

FAST – NUCES



Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 12 – Polymorphism and Abstract Classes

Objectives

- Abstract classes/ interfaces
 - Runtime polymorphism
 - Aggregation & Composition (Advance example with resizing)
 - Type casting (static_cast and dynamic_cast)
 - final class in OOP
-



Read carefully and attempt following task

Become a Pokémon Master

Welcome back, Trainers!

In this lab, you will extend your previous lab 11 implementation by applying advanced OOP concepts in C++.



Welcome to the world of Pokémon!

Instructions

- You are **allowed to use built-in string** in this lab

This lab is part 2 of lab 11.

Task 1: Pokémon as an Interface (Pure Abstract Class)

You will now convert the Pokemon class into a **pure abstract class**.

Key Instructions:

- Declare at least one function as **pure virtual** (= 0)
- showInfo() must be pure virtual
- You must NOT create objects of Pokemon
- All derived Pokémon classes must implement showInfo()

Important Design Rule:

- Keep Pokémon attributes (name, level) as **protected**

This allows derived classes (Fire, Water, etc.) to directly access them without getters/setters.

Task 2: Polymorphism with Pokémon Types

All Pokémon types:

- FirePokemon
- WaterPokemon
- ElectricPokemon
- GrassPokemon
- FlyingPokemon

What you will do:

- Each class adds its own attribute (same as previous lab)
- Override showInfo()
- Use a **base class pointer (Pokemon*)** to store different Pokémon types

Concept:

This demonstrates **runtime polymorphism**

Same function (showInfo) behaves differently depending on object type

Task 3: Trainer Hierarchy (Polymorphism)

You will reuse:

- Trainer
- GymTrainer
- GymLeader

Required Changes:

- Trainer must remain a **normal (non-abstract) class**
- Make showInfo() **virtual**
- Override it in all derived classes

Important Constraint:

- Declare GymLeader as final class: (class GymLeader final)

This prevents further inheritance.

Task 4: Relationships - Aggregation & Composition

Now we connect Trainers with Pokémon and Badges.

Aggregation: Trainer HAS Pokémon

Concept:

- Pokémon exist independently of Trainer
- Trainer only **references them**

Implementation Idea:

Inside Trainer:

```
Pokemon** pokemons;  
int pokemonCount;
```

addPokemon Function (Prototype)

```
void addPokemon(Pokemon* p);
```

How it works:

- Accepts a **pointer to any Pokémon type**
- Stores it in a dynamic array
- Supports **polymorphism** (different Pokémon types in same array)

Dynamic Resizing Logic:

Each time you add a Pokémon:

1. Create a new array of size count + 1
2. Copy old elements
3. Add new Pokémon at last index
4. Delete old array
5. Update pointer

6. Increment pokemonCount

This ensures:

- No fixed limit
- Flexible storage

Composition: Trainer HAS Badges

Concept:

- Badge belongs completely to Trainer
- Created and destroyed with Trainer

Inside Trainer:

Badge* badges;
int badgeCount;

addBadge Function (Prototype)

void addBadge(int id, string info);

Important Difference from Aggregation:

- You do NOT pass Badge object
- You pass **data**
- Badge is created inside the function

Dynamic Resizing (Same Concept):

- Increase array size by 1 each time
- Copy old data
- Add new badge
- Update badgeCount

Task 5: Maintaining Counts

Each Trainer must track:

- Total number of Pokémon
- Total number of Badges

Requirement:

Display these counts inside showInfo()

This helps verify:

- Aggregation working correctly
- Composition working correctly

Task 6: Type Casting in C++

Casting is used when working with **base class pointers**.

Static Cast

```
Derived* d = static_cast<Derived*>(basePtr);
```

Explanation:

- Checked at **compile time**
- Faster
- No safety check at runtime
- Can be dangerous if type is incorrect

Dynamic Cast

```
Derived* d = dynamic_cast<Derived*>(basePtr);
```

Explanation:

- Checked at **runtime**
- Safe (returns nullptr if cast fails)
- Requires **virtual functions** in base class

RTTI (Run-Time Type Information)

Dynamic casting uses **RTTI**.

What is RTTI?

- Mechanism to identify object type at runtime
- Enabled when using virtual functions
- Helps ensure safe casting

When to Use:

- Use `static_cast` when you are **sure of type**
- Use `dynamic_cast` when type is **uncertain**

Basic Syntax of RTTI

1. Using typeid

```
typeid(*ptr).name()
```

Explanation:

- `ptr` - base class pointer
- `*ptr` - actual object being pointed to
- `.name()` - returns type name

Task 7: Main Function – Complete Testing

You must test **ALL concepts together**.

Create Trainers

- Ash Ketchum (15)
- Misty (16)
- Gary Oak (16)

Create Gym Trainers

- Daisy - Cerulean Gym - Water
- Flint - Cinnabar Gym - Fire

Create Gym Leaders

- Brock - Pewter Gym - Rock - Boulder Badge
- Lt. Surge - Vermilion Gym - Electric - Thunder Badge

Create Professor Trainer

- Professor Willow
 - Age: 35
 - Field: Pokémon Behavior
 - Research Area: Field Research

Important Scenario

For Ash:

- Add **multiple Pokémon** using addPokemon()
- Use **different Pokémon types**
- This must demonstrate **polymorphism**

Example Pokémon for Ash:

- Pikachu - Electric (Voltage: 90)
- Charizard - Fire (Flame Power: 120)
- Bulbasaur - Grass (Growth Rate: 70)
- Squirtle - Water (Water Pressure: 80)
- Pidgeotto - Flying (Flight Speed: 85)

Add Badges to Ash:

- Thunder Badge
- Cascade Badge
- Boulder Badge

Using:

```
addBadge(int id, string info);
```

Example Badges for Ash:

- ID: 101 - Thunder Badge
- ID: 102 - Cascade Badge
- ID: 103 - Boulder Badge

Display Everything

For each Trainer:

- Basic Info
- All Pokémon (via polymorphism)
- All Badges
- Total counts

Task 8: Casting Demonstration

Scenarios to Test:

1. Store a GymLeader object in a Trainer*
2. Cast it back using:
 - static_cast
 - dynamic_cast
3. Use Trainer* for:
 - GymTrainer
 - GymLeader
 - ProfessorTrainer
4. Attempt safe casting using dynamic_cast
 - Check for nullptr

This ensures understanding of:

- Base vs derived relationships
- Safe vs unsafe casting

-----Happy Coding! 😊-----



Submission Instructions

- Submit your tasks on Google Classroom within given deadline
 - Submit one cpp file only.
 - Also submit screenshots of your task
 - “.cpp” is the actual file where all your code is saved.
 - Always submit .cpp files when asked for submission
-

